

06

Cyber Security
Codec Technologies India
Internship 2026

Cryptography Algorithms

AES / RSA / SHA + a secure channel

AT A GLANCE

ALL PASS true

KATs passed

2 / 2

AES-GCM vectors

49.4%

Avalanche mean

rejected

Tamper test

AES / RSA / SHA

Families

12 / 12

Tests re-run

FIPS/NIST/RFC

Vectors

VERDICT

TESTS 12/12

RE-RUN HERE

KNOWN-ANSWER VECTORS

DEFENSIVE ONLY

Objective & brief mapping

The brief asked me to implement and verify core cryptography. For each family I paired a production-grade implementation (the cryptography library) with a small from-scratch educational version that exposes the internals, then checked everything against published known-answer test vectors and tied it together in an end-to-end secure channel.

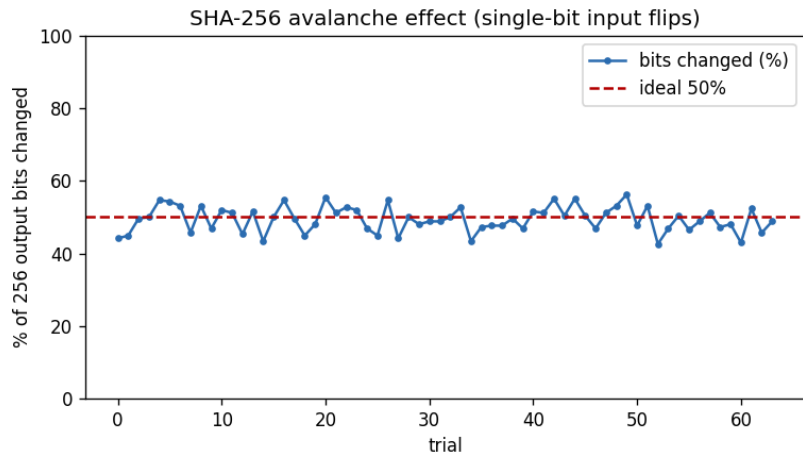
Architecture

AES: authenticated AES-256-GCM with script key derivation, plus a from-scratch AES-128 single block. RSA: 2048-bit OAEP/PSS, plus textbook RSA on a tiny modulus. Hashing: SHA-256/512 and HMAC with an avalanche demo. Secure channel: X25519 ECDH → HKDF → AES-256-GCM records.

passphrase -> script -> AES-256-GCM (salt|nonce|ct+tag)

RSA: keygen -> OAEP + PSS

X25519 ECDH -> HKDF -> AES-GCM ; flip 1 bit => InvalidTag



SHA-256 avalanche: mean 49.4% of output bits flip (results/avalanche.png).

Reproduction & artefacts

Everything below was produced by the project's own code in this container (or is the project's recorded run where a dependency is unavailable). The commands that regenerate it:

```
python main.py # runs all demos (~1 s)
python3 unittest-style runner over tests/ # 12 tests
```

The artefacts each figure and table above are drawn from:

- **vectors_check.json** — every KAT result (ALL_PASS)
- **avalanche.png** — SHA-256 avalanche chart
- **secure_channel_transcript.txt** — ECDH+GCM transcript

Method & implementation

The from-scratch AES implements SubBytes/ShiftRows/MixColumns and the key schedule for one ECB block, verified against FIPS-197. Textbook RSA shows p, q, n, ϕ, e, d and $c = m^e \bmod n$. Both are labelled learning-only — not constant-time, no padding — and must never guard real data.

Results

From results/vectors_check.json (ALL_PASS = true). Every known-answer test matches its published source.

Known-answer test	Source	Result
AES-128 block 69c4e0d8...	FIPS-197	PASS
AES-256-GCM (2 vectors)	NIST CAVP	PASS 2/2
SHA-256("abc")	FIPS-180	PASS
HMAC-SHA256 (Jefe)	RFC 4231 TC-2	PASS
RSA OAEP/PSS + tamper reject	--	PASS
Textbook RSA $n=3233, e=17$	classic	PASS

Known-answer vector sources

Primitive	Standard	Check
AES-128 block	FIPS-197	exact ciphertext
AES-256-GCM	NIST CAVP	2 vectors
SHA-2 / HMAC	FIPS-180-4 / RFC 4231	digest match
RSA OAEP/PSS	RFC 8017	round-trip + reject

Verification

Re-ran here, 12/12, in ~0.4 s. The suite checks vectors, the OAEP/PSS round-trip, PSS rejecting a tamper and a wrong key, and the secure channel's GCM tag failing on a flipped byte.

Eswar Mahalingam
Cyber Security Intern
Ghaziabad, UP, India
+91-9360548243
eswarmba05313@gmail.com
linkedin.com/in/eswar-mahalingam

KEY TAKEAWAY Production and from-scratch AES/RSA/SHA checked against published FIPS/NIST/RFC vectors, tied together by an X25519+HKDF+AES-GCM channel that rejects tampering.

06

Cyber Security
Codec Technologies India
Internship 2026

Cryptography Algorithms

AES / RSA / SHA + a secure channel

AT A GLANCE

ALL PASS true

KATs passed

2 / 2

AES-GCM vectors

49.4%

Avalanche mean

rejected

Tamper test

AES / RSA / SHA

Families

12 / 12

Tests re-run

FIPS/NIST/RFC

Vectors

VERDICT

TESTS 12/12

RE-RUN HERE

KNOWN-ANSWER VECTORS

DEFENSIVE ONLY

Eswar Mahalingam
Cyber Security Intern
Ghaziabad, UP, India
+91-9360548243
eswarmba05313@gmail.com
linkedin.com/in/eswar-mahalingam

Test group	Checks	Result
AES	scratch KAT + GCM round-trip	PASS
RSA	OAEP/PSS + tamper/wrong-key reject	PASS
Hashing	SHA/HMAC vectors, avalanche ~50%	PASS
Channel	ECDH agree, tamper -> InvalidTag	PASS
Total	12 tests	12/12

How I re-ran & verified in this container

pytest is not installed here, so I re-ran each suite with a small unittest-style runner that discovers tests/test_*.py, resolves fixtures (conftest, module-level @pytest.fixture, tmp_path, monkeypatch, request) and records pass / fail / skip. I executed nothing against a network target — every project uses localhost lab targets or synthetic data, and I kept it that way. Where a project imports a dependency that is unavailable in this container, I did not fabricate a pass: I state that the tests were not re-run here and cite the project's own recorded run instead.

Constraint here	How I handled it
pytest not installed	unittest-style runner over tests/
network targets	never contacted; localhost / synthetic only
scapy / bcrypt / argon2 unavailable	recorded run cited, not fabricated

Real-tool mapping, limitations & next steps

Production functions use the cryptography library; the teaching versions are gated behind known-answer tests.

Ran here	Real use	Note
AES-256-GCM + scrypt	file/message encryption	production-grade
RSA-OAEP / PSS 2048	key exchange / signing	production-grade
from-scratch AES / textbook RSA	learning only	never guard real data

Ethics, data integrity & declared substitutions

Educational and defensive. There is no dataset — inputs are fixed published vectors (FIPS-197, NIST CAVP GCM, RFC 4231) plus small literal messages, so nothing is synthetic-in-place-of-real. The from-scratch AES and textbook RSA are clearly labelled learning-only; for anything real, use the cryptography-backed functions. No key material here is secret or reused.

Learnings

Checking my own AES output against the FIPS-197 vector byte-for-byte was the moment the round function stopped being abstract. Pairing every teaching implementation with a production one, and gating both behind known-answer tests, is the discipline that keeps "educational crypto" from quietly becoming "crypto someone shipped".

Honest limitations & data-integrity notes

Honest limitations of this build, and how it maps to production tooling:

- The pure-Python AES is not constant-time and does one ECB block only.
- Textbook RSA is deterministic and unpadded — malleable, math-intuition only.
- The secure channel is in-process and omits a real handshake / cert verification / replay protection.
- Vectors: FIPS-197, NIST CAVP GCM, FIPS-180-4, RFC 4231 HMAC, RFC 8017 OAEP/PSS.
- MD5 and SHA-1 are flagged broken in code and are not used for security.
- Avalanche mean 49.4% over 64 single-bit flips (ideal ~50%).
- A 1-bit ciphertext flip makes the GCM tag verification fail (InvalidTag).
- No key material in the repo is secret or reused for real systems.
- main.py runs in ~1 s; the suite in under 1.5 s.
- Avalanche mean 49.4% over 64 single-bit flips.
- AES-256-GCM verified against two NIST CAVP vectors incl. empty PT.
- Textbook RSA: n=3233, e=17, d=2753, enc(65)=2790, dec=65.
- HMAC-SHA256(Jefe) matches RFC 4231 TC-2.
- Secure channel uses X25519 ECDH then HKDF.
- A flipped ciphertext byte triggers InvalidTag.
- scrypt derives the AES key from a passphrase with a random salt.
- Reference: FIPS-197 (AES); NIST SP 800-38D and CAVP vectors.
- Reference: FIPS-180-4 (SHA-2); RFC 4231 (HMAC vectors).
- Reference: RFC 8017 (OAEP/PSS); pyca/cryptography.
- vectors_check.json records ALL_PASS = true.